

CMPU3010: Databases

WINDOW FUNCTIONS IN POSTGRESQL:
COMPARING, RANKING AND ANALYSING ROWS

Learning Outcomes

Explain	How a window function differs from GROUP BY.
Identify	The function, partition and ordering parts of a window expression.
Number	Number rows using ROW_NUMBER().
Rank	Rank rows using RANK() and DENSE_RANK().
Access	Access earlier and later rows using LAG() and LEAD().
Use	Use COUNT(), SUM() and AVG() without removing individual rows.
Decide	Whether a window requires: PARTITION BY or ORDER BY or Both
Interpret	NULL values returned by LAG() and LEAD().

When joins and subqueries become awkward

Some questions require each row to be compared with other rows in the same result:

- Where does this service rank compared with other services?
- What was the value in the preceding row?
- What is the next appointment in a sequence?
- How many related appointments are there while still displaying each appointment?
- How does an appointment's fee compare with the average for its service?
- These questions can be answered using joins or subqueries, but the resulting SQL may become complex.

Window functions are designed for this type of calculation.

Our Example

The inner query runs in the context of the current row and searches for the latest earlier appointment belonging to the same patient.

```
SELECT
    a.patient_id,
    a.appointment_date,
    (
        SELECT MAX(previous.appointment_date)
        FROM appointment previous
        WHERE previous.patient_id =
a.patient_id
            AND previous.appointment_date <
a.appointment_date
    ) AS previous_appointment
FROM appointment a
ORDER BY
    a.patient_id,
    a.appointment_date;
```

FOR each appointment

Search through all appointments again

Find earlier appointments for the same patient

Select the latest of those earlier appointments

END FOR

Sort the results by patient and appointment date

Window Functions

A window function expresses the same relationship more directly

The correlated subquery and window function may produce the same result, but their execution plans differ. We will examine query execution and performance later in the module.

```
SELECT patient_id, appointment_date,  
       LAG(appointment_date)  
       OVER  
         ( PARTITION BY patient_id ORDER BY  
           appointment_date ) AS previous_appointment  
FROM appointment  
ORDER BY patient_id, appointment_date;
```

Read all appointments once

Sort the appointments by:

1. patient
2. appointment date

FOR each patient

Look at the previous row in that patient's ordered appointments

END FOR

Window Functions

Window functions allow us to:

- Compare a row with earlier or later rows.
- Rank and number rows.
- Calculate totals and averages without removing detail rows.
- Perform calculations within groups while keeping every original row.

A window function calculates across a set of related rows while preserving the individual rows in the result.

What we are
covering
today...

In the lab, you used:

- ROW_NUMBER() to number a patient's appointments.
- LAG() to find a patient's previous appointment.
- RANK() to rank services by appointment count.

Today we are going to look at:

- Explain how those functions work
- And look at
 - LEAD()
 - COUNT() OVER()
 - SUM() OVER()
 - AVG() OVER()

GROUP BY v Window Functions

```
SELECT
    patient_id,
    COUNT(*) AS appointment_count
FROM appointment
GROUP BY patient_id;
```

This produces one row for each patient.

Individual appointment rows are no longer displayed.

What if we want to include the appointment date?

Or create a report which includes the appointment id for each appointment and the overall total number of appointments for each patient?

GROUP BY and detail rows

With GROUP BY, every selected column must either:

Appear in the GROUP BY, or

Be used inside an aggregate function.

GROUP BY v Window Functions

```
SELECT  
  
    appointment_id,  
    patient_id,  
    appointment_date,  
    COUNT(*) OVER (  
  
        PARTITION BY patient_id  
  
    ) AS appointment_count  
FROM appointment;
```

A window function performs a calculation across related rows without combining those rows into one result.

GROUP BY v Window Functions

GROUP BY	Window function
Combines related rows	Retains individual rows
Produces one row per group	Produces a result for every row
Used for summary results	Adds group information to detail rows

General window function syntax: OVER()

A window expression has the following general form:

```
function_name(column_name)
OVER (
    PARTITION BY
    grouping_column
    ORDER BY ordering_column
)
```

The OVER() clause defines the rows available to the window function.

- It retains every relevant row and places the calculation (aggregate) beside each one.

General window function syntax: PARTITION BY

A window expression has the following general form:

```
function_name(column_name) OVER  
(  
    PARTITION BY grouping_column  
    ORDER BY ordering_column  
)
```

PARTITION BY divides the rows into separate windows

- It creates a window for each instance of each relevant entity
- The calculation (aggregation) is performed independently within each window.
- A partition does not combine the rows. It only identifies which rows should participate in each calculation.

Without PARTITION BY PostgreSQL will treat the full resultset from the select as the partition and display the **total number of appointments in the table** beside every row.

General
window
function
syntax:
ORDER BY

Some window functions need rows to be placed in a meaningful sequence.

For example:

```
ROW_NUMBER() OVER (  
    PARTITION BY patient_id  
    ORDER BY appointment_date  
)
```

The window ORDER BY determines how each patient's appointments are numbered.

It does not necessarily control the final display order.

There are two different uses of ORDER BY:

General window function syntax: ORDER BY

Some window functions need rows to be placed in a meaningful sequence.

For example:

```
ROW_NUMBER() OVER (  
    PARTITION BY patient_id  
    ORDER BY appointment_date  
)
```

The window ORDER BY determines how each patient's appointments are numbered.

It does not necessarily control the final display order.

There are two different uses of ORDER BY:

Location	Purpose
Inside OVER()	Controls the window calculation
At the end of the query	Controls how the results are displayed

General window function syntax: ORDER BY

```
SELECT
    appointment_id,
    patient_id,
    appointment_date,
    appointment_time,
    ROW_NUMBER() OVER (
        PARTITION BY patient_id
        ORDER BY appointment_date,
        appointment_time, appointment_id
    ) AS appointment_number
FROM appointment
ORDER BY
    patient_id,
    appointment_date,
    appointment_time,
    appointment_id;
```

Location	Purpose
Inside OVER()	Determines the order used to calculate the row numbers
At the end of the query	Determines the order in which the result rows are displayed

General
window
function
syntax:
ORDER BY

```
SELECT
    appointment_id,
    patient_id,
    appointment_date,
    appointment_time,
    RANK() OVER (
        PARTITION BY
patient_id
        ORDER BY
appointment_date) AS
appointment_rank
FROM appointment
ORDER BY
    patient_id,
    appointment_id,
    appointment_time,
    appointment_date;
```

Ties are determined by **all the expressions inside the window's ORDER BY**. The final ORDER BY only controls how the results are displayed

General
window
function
syntax:
ORDER BY

```
SELECT
    appointment_id,
    patient_id,
    appointment_date,
    appointment_time,
    RANK() OVER (
        PARTITION BY patient_id
        ORDER BY appointment_date,
                appointment_time,
                appointment_id)
AS appointment_rank
FROM appointment
ORDER BY
    patient_id,
    appointment_date,
    appointment_time,
    appointment_id;
```

Ties are determined by **all the expressions inside the window's ORDER BY**. The final ORDER BY only controls how the results are displayed

Element	Purpose	Example
Window function	Specifies the calculation to perform	ROW_NUMBER(), RANK(), SUM()
OVER()	Identifies the calculation as a window calculation	RANK() OVER (...)
PARTITION BY	Divides the rows into separate groups. The calculation restarts for each group	PARTITION BY patient_id
Window ORDER BY	Sets the order used by the window calculation. For ranking functions, it also determines which rows are tied	ORDER BY appointment_date
Additional ordering columns	Resolve ties by providing a more precise order	ORDER BY appointment_date, appointment_time
Unique tie-breaker	Ensures that every row has a distinct position	ORDER BY appointment_date, appointment_time, appointment_id
Alias	Gives the calculated column a meaningful name	AS appointment_rank
Final query ORDER BY	Controls how the completed results are displayed; it does not change the window calculation	ORDER BY patient_id, appointment_date

General window function syntax

Numbering and Ranking Functions

ROW_NUMBER() assigns a different sequential number to every row

- ROW_NUMBER() performs the calculation
- PARTITION BY patient_id restarts the numbering for each patient
- ORDER BY appointment_date, appointment_time, appointment_id determines the numbering order.
- Every row receives a different number.
- The numbering begins at 1.
- The numbering restarts when the partition changes.

RANK() assigns a position to each row based on its ordering values.

- RANK() performs the ranking calculation.
- PARTITION BY patient_id restarts the ranking for each patient.
- ORDER BY appointment_date determines the ranking order.
- Rows with the same ordering value receive the same rank.
- The ranking begins at 1.
- After a tie, the next rank contains a gap.
- The ranking restarts when the partition changes.

DENSE_RANK()

Equal values receive the same rank. The next rank is not skipped.

Number and Ranking Functions – Combined Example

```
SELECT
    p.first_name, p.last_name, a.appointment_id, a.appointment_date,
    a.appointment_time,

    ROW_NUMBER() OVER (
        PARTITION BY a.patient_id
        ORDER BY
            a.appointment_date, a.appointment_time, a.appointment_id
    ) AS row_number,

    RANK() OVER (
        PARTITION BY a.patient_id
        ORDER BY a.appointment_date
    ) AS appointment_rank,

    DENSE_RANK() OVER (
        PARTITION BY a.patient_id
        ORDER BY a.appointment_date
    ) AS dense_appointment_rank

FROM patient p
JOIN appointment a
    ON p.patient_id = a.patient_id

ORDER BY
    a.patient_id, a.appointment_date, a.appointment_time, a.appointment_id;
```

LAG()

```
SELECT
    patient_id,
    appointment_date,
    LAG(appointment_date)
    OVER (
        PARTITION BY patient_id
        ORDER BY
            appointment_date,
            appointment_time,
            appointment_id
    ) AS previous_appointment
FROM appointment;
```

LAG() accesses a value from an earlier row in the window.

- The column inside LAG() is the value to retrieve: LAG(appointment_date)
- The partition identifies the rows that belong together: PARTITION BY patient_id
- The ordering defines “previous”: ORDER BY appointment_date, appointment_time, appointment_id

Why does LAG() return NULL?

Consider the first appointment in a patient's partition.

The first row has no preceding row in that partition.

Therefore LAG(appointment_date) returns NULL.

This is not an error. It means that no previous value exists within the window.

Patient	Date	Previous date
Patient A	10 February	NULL
Patient A	3 March	10 February
Patient A	21 April	3 March

LEAD()

```
SELECT
    patient_id,
    appointment_date,
    LEAD(appointment_date) OVER (
        PARTITION BY patient_id
        ORDER BY
            appointment_date,
            appointment_time,
            appointment_id
    ) AS next_appointment
FROM appointment
ORDER BY
    patient_id,
    appointment_date;
```

LEAD() accesses a value from a later row.

- The column inside LEAD() is the value to retrieve: LEAD(appointment_date)
- The partition identifies the rows that belong together: PARTITION BY patient_id
- The ordering defines “next”: ORDER BY appointment_date, appointment_time, appointment_id

Why does
LEAD() return
NULL?

The final row has no later row in the partition, so LEAD() returns NULL.

Patient	Date	Next date
Patient A	10 February	3 March
Patient A	3 March	21 April
Patient A	21 April	NULL

LEAD() v LAG()

The syntax is nearly identical:

```
LAG(column_name) OVER (  
  
    PARTITION BY group_column  
  
    ORDER BY sequence_column  
  
)
```

```
LEAD(column_name) OVER (  
  
    PARTITION BY group_column  
  
    ORDER BY sequence_column  
  
)
```

What changes is the direction in which PostgreSQL looks

Function	Direction	Typical question
LAG()	Backwards	What was the previous value?
LEAD()	Forwards	What is the next value?

Aggregates can be window functions

You already know aggregate functions:

- COUNT()
- SUM()
- AVG()

You previously used these functions with GROUP BY.

These can also be used as window functions by adding OVER().

- COUNT(*) OVER (...)
- SUM(column_name) OVER (...)
- AVG(column_name) OVER (...)

This retains every source row.

COUNT() OVER()

Show the number of appointments for each service without removing the individual appointments:

```
SELECT
    appointment_id,
    service_id,
    appointment_date,
    COUNT(*) OVER (
        PARTITION BY service_id
    ) AS service_appointment_count
FROM appointment
ORDER BY
    service_id,
    appointment_date;
```

The value is repeated because all rows in the same service partition have the same count.

SUM() OVER()

Assume the query joins an appointment to its service:

```
SELECT
    a.appointment_id,
    a.patient_id,
    s.service_name,
    s.standard_fee,
    SUM(s.standard_fee) OVER (
        PARTITION BY a.patient_id
    ) AS total_standard_fees
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
WHERE a.status = 'Completed'
ORDER BY
    a.patient_id,
    a.appointment_date;
```

For each completed appointment, the query displays the total standard fees associated with all of that patient's completed appointments.

The appointment rows remain visible.

Important point:

- The column being added does not need to be the partitioning column.

Here:

- standard_fee is being added.
- patient_id determines which rows are added together.

AVG() OVER()

Calculate the average service fee associated with each patient's completed appointments:

```
SELECT
    a.appointment_id,
    a.patient_id,
    s.service_name,
    s.standard_fee,
    AVG(s.standard_fee) OVER (
        PARTITION BY a.patient_id
    ) AS average_standard_fee
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
WHERE a.status = 'Completed'
ORDER BY
    a.patient_id,
    a.appointment_date;
```

The result may contain several decimal places so you could use ROUND.

AVG() OVER()

The result may contain several decimal places so you could use ROUND.

```
SELECT
    a.appointment_id,
    a.patient_id,
    s.service_name,
    s.standard_fee,
    ROUND(
        AVG(s.standard_fee) OVER (
            PARTITION BY a.patient_id
            ),2) AS average_standard_fee
FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
WHERE a.status = 'Completed'
ORDER BY
    a.patient_id,
    a.appointment_date;
```

Combining Functions

Each function uses the same patient partition but performs a different calculation

```
SELECT
    a.appointment_id, a.patient_id, a.appointment_date,
    s.standard_fee,
    COUNT(*) OVER (
        PARTITION BY a.patient_id
    ) AS completed_appointment_count,

    SUM(s.standard_fee) OVER (
        PARTITION BY a.patient_id
    ) AS total_standard_fees,

    ROUND(
        AVG(s.standard_fee) OVER (
            PARTITION BY a.patient_id
        ), 2 ) AS average_standard_fee

FROM appointment a
JOIN service s
    ON a.service_id = s.service_id
WHERE a.status = 'Completed'
ORDER BY
    a.patient_id, a.appointment_date;
```

When is
window
ORDER BY
required?

Usually Required

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- LAG()
- LEAD()

Not Usually Required

COUNT(*) OVER (PARTITION BY patient_id)
SUM(standard_fee) OVER (PARTITION BY
patient_id)
AVG(standard_fee) OVER (PARTITION BY
patient_id)

Adding an ORDER BY to SUM() or COUNT() changes the meaning and may create a cumulative calculation. It should not be added unless a cumulative result is required.



Join at:
vevox.app

ID:
127-632-361



Final Quick Knowledge Check
